

SkyJet Flight Recovery

Architecture Board

One document to understand the **whole system and every feature**. A self-service experience that lets a disrupted passenger **rebook, refund, or get an answer in under 30 seconds** — with complex cases handed to a human, full context attached.

PRODUCT	SHAPE	STATUS	UPDATED
Self-service IRROPS recovery	Modular monolith · Next.js 16	Deployable MVP · Vercel- ready	2026-07-03

BOARDS IN THIS DOCUMENT

- 01 Whole-system architecture

03 Eligibility rules engine

05 Data model & booking state machine

07 Scale & non-functional story
- 02 Golden-path sequence

04 Disruption Assistant · RAG

06 Feature map (P0–P3)

01 Whole-system architecture

A **modular monolith** — one deployable Next.js process with clean domain seams, so any layer could later split into its own service without a rewrite. The request flows top → bottom through six layers. High-frequency, low-risk decisions are automated; high-risk, low-frequency ones get a warm agent handoff.

■ Entry ■ Client / API / Data ■ Eligibility engine ■ Assistant / RAG

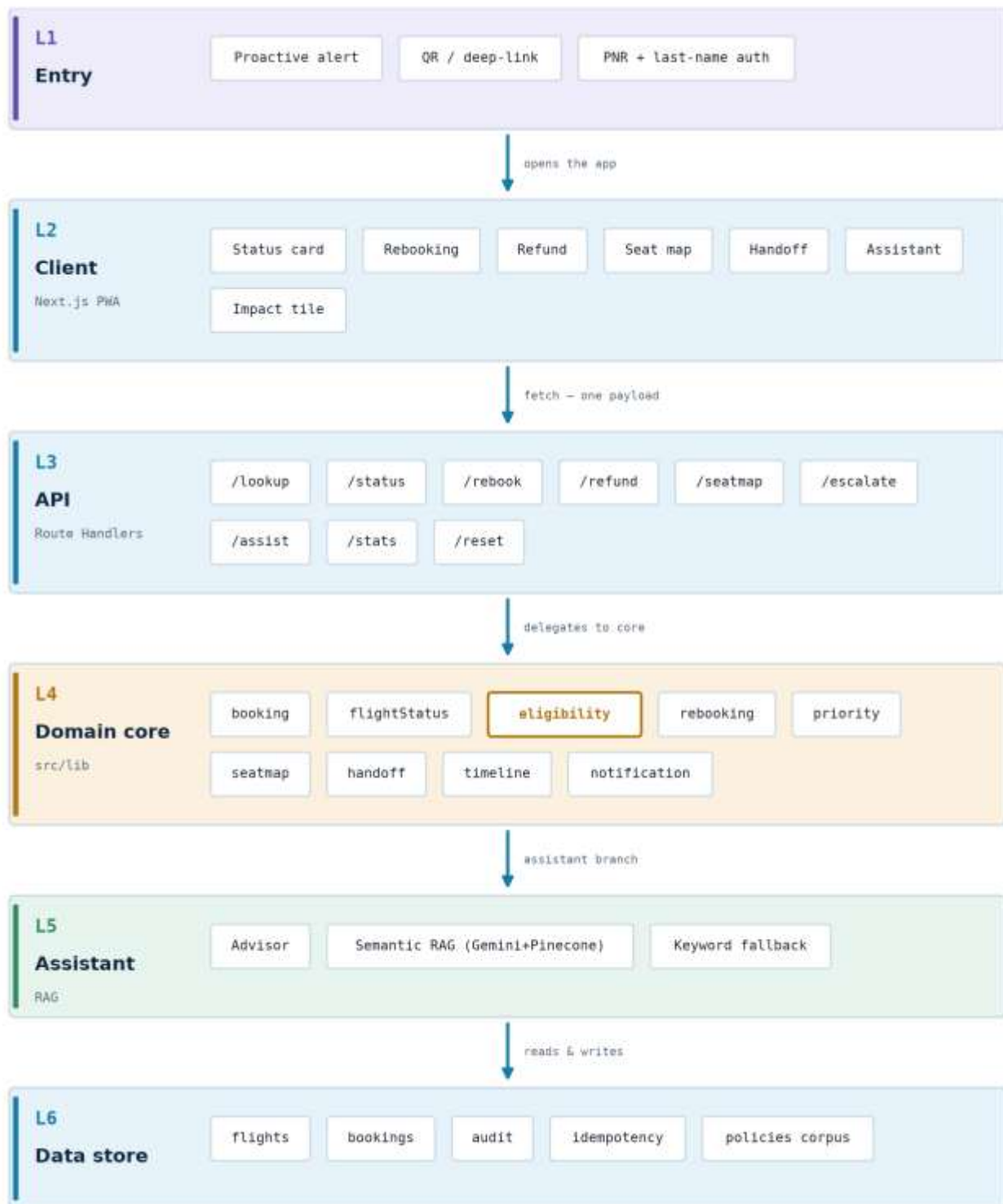


Fig 1 – Six layers, request flowing top → bottom. Eligibility (L4) highlighted.

Why this shape

- **Modular monolith:** one deployable, one process, zero infra to provision — runs identically on a laptop and on Vercel. The six clean module seams make “microservices-ready” credible without a rewrite.

- **Explain, don't just decide:** every eligibility result carries a human reason and a DGCA rule citation — the same facts ground both the UI and the assistant, so the product defends every answer aloud.
- **Spike-survivable writes:** idempotency-key replay + optimistic version + revalidate-against-fresh-options mean a double-tap or a stale tab can never double-book during a disruption surge.

Cross-cutting — the spike-survival story

Idempotent, race-safe writes Key replay + revalidate-vs-fresh-options — a double-tap can never double-book.	Optimistic concurrency Every booking carries a version; a stale tab is rejected.
Append-only audit Every mutation logs action + before/after; powers the impact tile.	Stateless handlers No per-request state → scale flat behind a load balancer.
Typed contracts Zod at every boundary; typed { error } responses.	Deterministic time All formatting pinned to Asia/Kolkata — the demo never drifts.
Guarded admin surface Rate-limit + admin token on reindex / flight controls.	Async-ready notification The mock alert is the seam for a queue + outbox in production.

Module responsibilities (L4 domain core)

Module	File(s)	Responsibility	Deployable as
booking	store.ts · lookup/route.ts	PNR + surname lookup; assemble the booking view	booking-service
flightStatus	eligibility.ts · store.ts	Classify CANCELLED / LONG_DELAY; expose route inventory	flight-status-service
eligibility ★	eligibility.ts	DGCA rules engine — refund / rebook / compensation / duty-of-care with reasons + rule refs	eligibility-service (pure)
rebooking	service.ts · rebook/route.ts	Scored alternatives + recommended pick; idempotent rebook; boarding pass	rebooking-service
priority / seatmap	priority.ts · seatmap.ts	Passenger priority rank; priority-aware seat map build + allocation	seat-service
handoff	service.ts · escalate/route.ts	Detect out-of-policy cases; package the warm-handoff context pack	handoff-service
notification	UI + mock	Render the proactive alert that would be sent (deep link + held seat)	notification-service (async)

02 Golden-path sequence

The ~90-second demo, as a flow: proactive alert → identified status card → recommended rebook → boarding pass, with the refund and agent-handoff branches, closing on the impact tile.

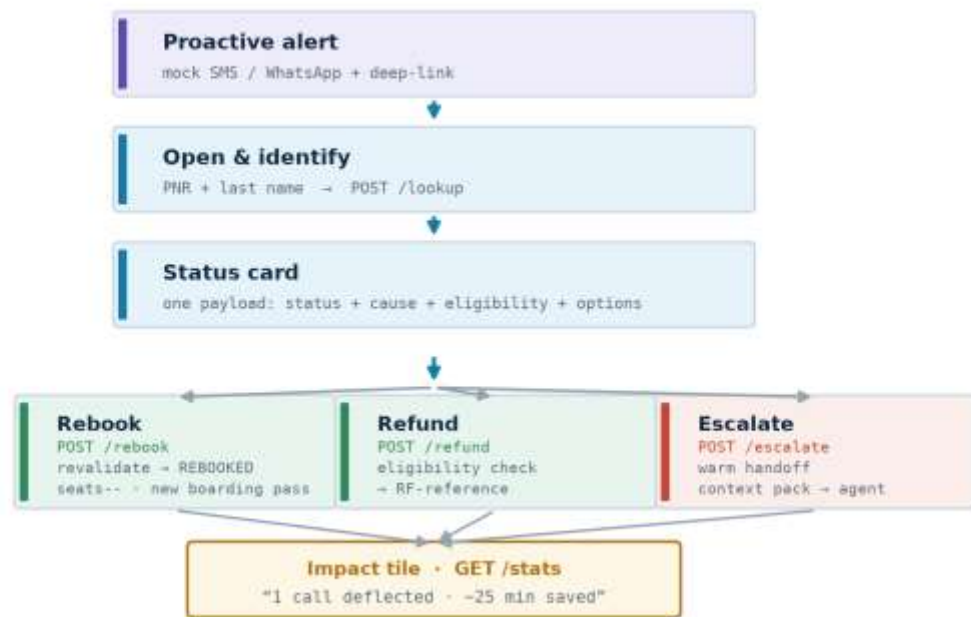


Fig 2 – One identify call, then the passenger's choice branches, all converging on the impact tile.

1	Proactive alert Passenger Mock SMS/WhatsApp with a deep-link — the airline reaches out before the passenger calls.
2	Open & identify Passenger → App → /lookup PNR pre-filled from the link; passenger adds last name. One call returns status + cause + eligibility + options.
3	See the status App “Cancelled · Weather” with a plain-English explanation of exactly what they’re owed.
4a	Rebook /rebook (idempotencyKey) Tap the recommended option → revalidate against fresh inventory → REBOOKED, seats--, audit → new boarding pass (< 30s).
4b	Refund /refund (idempotencyKey) Eligibility check → issue RF-reference → explainable decision. No payment integration — reference only.

4c	Escalate /escalate Minor / medical / group / no valid option → package warm-handoff context so the agent never asks the passenger to repeat.
5	Impact tile /stats “1 call deflected · ~25 min saved”, derived from the audit log.

03 Eligibility rules engine — the crown jewel

[src/lib/eligibility.ts](#) — pure, deterministic, unit-tested. The defensible rule, cross-validated by DGCA (India) and Delta: **the cause of the disruption drives entitlement.**

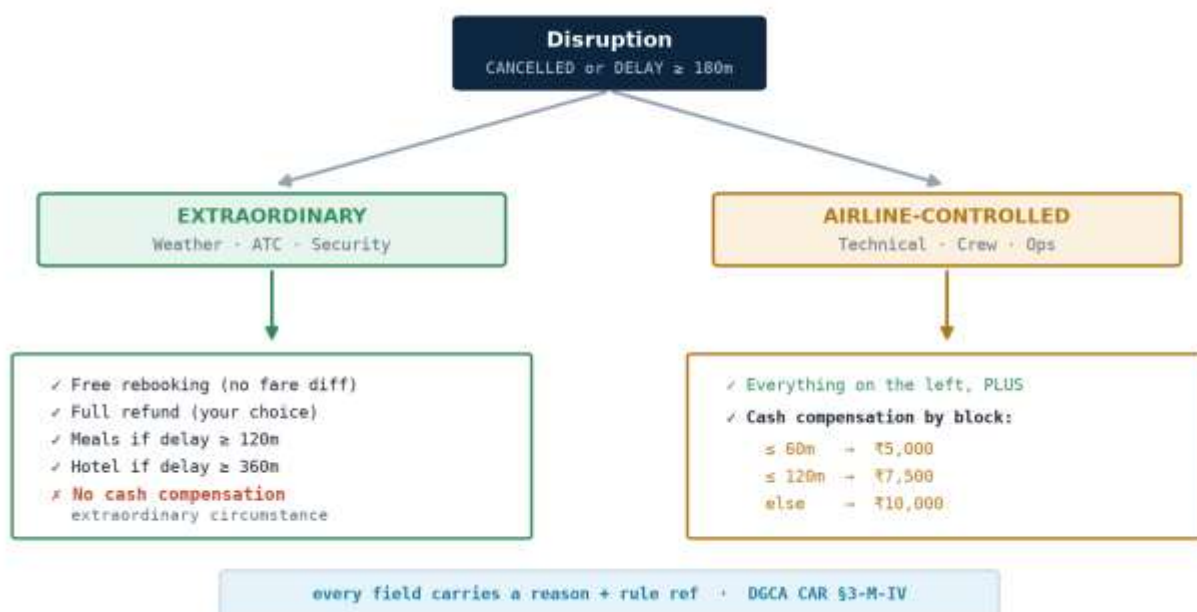


Fig 3 – Cause drives entitlement: extraordinary vs airline-controlled, each with its rule reference.

Trigger: disrupted = CANCELLED, or DELAYED ≥ 180 min. Every field carries a human-readable reason and a rule citation (DGCA CAR §3-M-IV) — the direct answer to competitor tools criticised as opaque.

Rebooking recommendation

Each candidate starts at 100: −30 next-day, −5/h later (cap −40), +10 if within 2h of the original time-of-day. The top score is flagged recommended with a plain-English reason (“Same day · 3h later · direct”). Options are recomputed live from inventory, so a shown option is never stale.

Escalation triggers

Special flags (unaccompanied minor, medical, pet, group, partner ticket), party size > 4, or no valid self-service rebooking → route to an agent with context attached.

04 Disruption Assistant · RAG

A layered design where every layer is grounded — the model can explain policy but never invent it, and the demo survives any vendor outage. /api/assist walks the ladder top-down and stops at the first tier that answers.



/api/assist walks the ladder top-down, stops at the first tier that answers — every tier grounded & cited

Fig 4 — The fallback ladder: stop at the first tier that answers; every tier grounded and cited.

- **Tier 0 · Advisor** — **deterministic advice** — “Should I refund or rebook?” → turns this passenger’s own eligibility + live rebooking options into a recommendation. The LLM (when configured) only rephrases it.
- **Tier 1 · Semantic RAG** — **Gemini + Pinecone** — Gemini embeds the question → Pinecone finds the top policy clauses → Gemini Flash writes a short answer using only those clauses + verified eligibility facts → citations attached. Returns null on outage/irrelevance.
- **Tier 2 · Keyword fallback** — **always on** — If embeddings/Pinecone are unconfigured: a deterministic tokenise → synonym-expand → score engine answers over the policy corpus, still cited. So it can never dead-air.

05 Data model & booking state machine

Runtime uses an in-memory seeded store (src/lib/store.ts) that mirrors prisma/schema.prisma field-for-field — including the optimistic-version and idempotency-key columns, so the production swap is mechanical.

- **Flight** — flightNo · origin/destination + cities · departure/arrival (ISO UTC) · durationMin · status (SCHEDULED/CANCELLED/DELAYED) · cause · delayMinutes · seatsAvailable. Indexed on (origin, destination, status, departure).
- **Passenger** — name · masked email · tier (STANDARD/SILVER/GOLD).
- **Booking** — ref (PNR, PK) · flightId (+ optional connectingFlightId) · status · fareClass · farePaid (INR) · specialFlags[] · partySize · mutation trail (rebookedFlightId, refundReference, handoffReference) · version (optimistic concurrency).
- **AuditEntry** — bookingRef · action (LOOKUP/REBOOK/REFUND/ESCALATE/ACKNOWLEDGE) · detail · before/after · idempotencyKey (unique) for race-safe, replayable writes.

Booking lifecycle

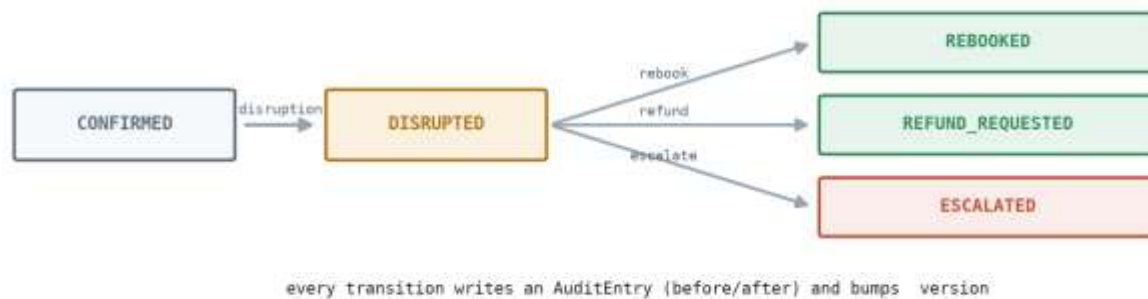


Fig 5 – A disruption moves CONFIRMED → DISRUPTED; the passenger's choice lands in one terminal state.

Every transition writes an AuditEntry (before/after) and bumps version — the basis of race-safe writes and the impact tile.

06 Feature map (P0–P3)

What ships in the MVP, what's built beyond it, and what's deliberately deferred — deferring on purpose is a scored signal of product judgment.

P0 – MVP core	P1 – Shipped beyond core	P2 – Credible on paper	P3 – Deliberately deferred
· Identify booking (PNR + last name) · Explainable status + cause · Scored rebooking + recommended pick · Refund eligibility + reference · Warm agent handoff with context · Impact tile (calls deflected)	· Proactive alert + QR deep-link · Priority-aware seat map · Grounded Disruption Assistant (RAG) · Idempotent, race-safe writes · Flight timeline view	· Prisma/Postgres schema (1:1) · Async notification seam (queue/outbox) · Cached flight status + read replicas · Admin reindex + flight controls	· Real payments · Real auth / SSO / OTP · Live inventory + interline/award · Loyalty redemption · multi-language · Predictive pre-disruption alerts · Cost/OR-optimized selection

07 Scale & non-functional story

Disruptions are traffic spikes — everyone asks the same three questions at once. The design answers that; each MVP mechanism maps to a production shape.

Concern	In this MVP	Production mapping
Statelessness	Route handlers hold no state; store is a module singleton	Stateless services behind a load balancer (ECS/Lambda)
Read spikes	flightStatus reads are cheap; route index modelled	Cache flight status (Redis/CDN); read replicas
Race-safe writes	Idempotency replay + optimistic version + revalidate	Same keys enforced by a DB unique constraint

Async fan-out	Notification is a mock “would-send”	Queue (SQS/Kafka) + outbox + DLQ
Auditability	Every mutation writes an AuditEntry (before/after)	Append-only audit table; PII redaction
Typed contracts	Zod at every boundary; typed { error }	RFC-7807 problem+json

Key assumptions

- Mock data — realistic, deterministic seed pinned to a fixed disruption day in IST.
- Auth = PNR + last name (OTP / IdP is future work).
- No payments — refund and compensation issue a reference number only.
- Airline-caused disruption $\Rightarrow$ no fare difference on rebooking.
- Thresholds: long delay 180 min · meals $\geq$ 120 min · hotel $\geq$ 360 min · compensation ₹5,000 / ₹7,500 / ₹10,000 by block time.
- In-memory store resets on cold start / POST /api/reset; production is the Prisma/Postgres schema.

SkyJet Flight Recovery · Architecture Board Next.js 16 · React 19 · TypeScript · Zod · Vitest · Vercel-ready

22North Product Engineering Challenge 2026 · Challenge 1